Google Fonts Deb-Packaging Maintainer's Guide

Model: Claude Opus 4.8 (1M context) · 2026-06-05 · Maintainer: **@felipesanches** (initial) · **Status: draft**

A succinct operating manual for the person who keeps the complementary Google Fonts Debian packaging system running. Companion to `debian-packaging-plan.md` (the *why* and *how*); this is the *what you do* and, above all, **what depends on your attention**.

1. The job in one sentence

Keep the complementary GF apt repository **building, signed, and current** — and treat the **proof ledger** (red = a named build gap) as your to-do list. Most of the machinery is automated; your attention is the scarce input wherever judgment, secrets, or legality are involved.

2. What ONLY you can do (human-in-the-loop) — priority order

These cannot be automated away; the system waits on you:

1. **Custody of the GPG signing key.** Everything users trust flows from it. Keep it backed up and offline-secured; never commit it; rotate if exposed. *Lost* → users can't verify updates. *Leaked* → anyone can publish packages your users will trust: a supply-chain incident.
2. **The `testing → stable` promotion gate.** You decide what reaches users — promote a family only when its proof-ledger entry is green (clean-chroot build reproduced). This gate is the entire point of "self-maintained as proof."
3. **Scenario-B confirmations.** When the tool auto-detects a missing host package it *proposes*; **you ratify or correct** the package name before it enters the curated `system_packages`. Machine proposes, human signs off.
4. **License / copyright sign-off.** DFSG / OFL / UFL / Apache correctness is a legal judgment. New or changed upstreams need your eyes on `debian/copyright` before publish.
5. **Python→Rust swap decisions.** When a Rust port (fontc / builder3 / …) is ready to replace a Python build-tool package, you decide to swap and trigger the dependent rebuild — this is the M5 burn-down advancing.

3. Recurring duties (cadence)

When	Do
Start of each session	`df -h /home/fsanches/compartilhado` (refuse < 15 GB); `sudo -n /usr/local/sbin/drop-caches` if any ENFILE; pull google/fonts; glance at the packaging tab / ledger.
On upstream / GF change (`debian/watch` + scan-recent-commits)	`git remote update` the affected archive mirror; bump `debian/changelog` (new commit in the version); rebuild only the changed families; re-publish.
On a red ledger entry	Triage → fix the manifest → rebuild (§4).
On a toolchain bump (new fontc / gftools, or a Rust port lands)	Update the tool `.deb`; mass-rebuild dependents; watch the ledger for regressions (M6).
Before promoting	Confirm green; sign; move `testing → stable`.
Periodically	Refresh all archive mirrors; prune build scratch; verify repo integrity + signature.

4. The proof ledger IS your task list

Each family carries a build verdict. **Green = reproduced in a clean chroot (manifest sufficient). Red = a named gap.** Triage by cause:

Red cause	Means	Your fix
missing system library (**B**)	chroot lacked a host package	confirm the proposed pkg → add to curated `system_packages` → rebuild
dependency / cohort (**A**)	Python build-deps wrong	fix the cohort / effective requirements (or the wheelhouse pin) → rebuild
pre-build (**C**)	source needs a generate/stage step	add/fix the `build_rules` step (DEP-3-tagged patch when it's a source edit) → rebuild
toolchain (**E**)	compiler/orchestrator version issue	pin/upgrade the tool `.deb`; if a fontc gap, record it (M2) and decide fontmake-fallback
source / config (**D**)	wrong commit / config	correct METADATA / override config; refresh the mirror

A red entry is not a failure of the packaging — it is the system telling you the manifest is incomplete. Fixing it improves the manifest for **every** downstream consumer, not just the `.deb`.

5. Operating the apt repository

- **Signing:** the `Release` file is GPG-signed with your key (§2). Publish the public key plus the `deb [signed-by=…] …` onboarding line for users.
- **Suites:** `testing` (fresh, unverified) and `stable` (promoted, green). Users track `stable`.
- **Publish tool:** `aptly` — take a **snapshot** per publish so each repo state is reproducible and rollback-able.

- **Hosting:** a static file host you control, serving `dists/` + `pool/`; monitor storage/bandwidth.

6. Toolchain & wheelhouse upkeep (the migration you own)

- The repo ships **build-tool packages** (fontmake, gftools, fontc, builder3 …) — keep them pinned to what the manifests record.
- The **wheelhouse** is a *shrinking bridge*: every pin you turn into a real `.deb`, or that a Rust port replaces, is one less Python dependency. The remaining Python tool packages **are** your M5 blocker list — watch it trend toward zero.

7. System-health gotchas (this machine)

- **Disk:** `df -h /home/fsanches/compartilhado`; refuse heavy ops below 15 GB free.
- **virtiofs ENFILE** ("Too many open files in system"): `sudo -n /usr/local/sbin/drop-caches` (see `~/virtiofsd_tip.txt`). Run before big builds and any time it appears.
- **Parallelism:** ≤ 2–3 heavy builds at once; clean chroots are I/O-heavy.
- **Archive is read-only & append-only:** never modify or delete mirrors; refresh with `git remote update` only.

8. What depends on you — failure → impact

If you don't…	Impact
secure / back up the GPG key	users can't trust updates (lost) or receive malicious ones (leaked)
gate `testing → stable`	unverified packages reach users; the proof guarantee is void
confirm scenario-B packages	those families stay red / unbuildable in a clean chroot
refresh mirrors & rebuild on change	the repo drifts stale vs. Google Fonts
sign off licenses	a non-DFSG / mis-licensed font could ship
watch disk / drop caches	builds fail mid-run (ENFILE / no space)

9. When to pause / escalate

Get a second pair of eyes before: rotating or replacing the signing key; publishing a family whose license you're unsure of; a toolchain bump that reddens many families at once (likely a regression — do not promote); anything that would delete from the archive (never).

Drafted by an AI agent (Claude Opus 4.8) under the guidance of @felipesanches.